UNION, CTEs, and Recursion

By Utkarsh Sahu, 28 Sep 2025

UNION and UNION ALL

`UNION` and `UNION ALL` are set operators used to combine the result sets of two or more `SELECT` statements. The columns in the `SELECT` statements must have similar data types and be in the same order.

When and How to Use

- **`UNION`**: Combines result sets and **removes duplicate rows**. Use it when you only want unique records from the combined queries. Because it has to check for duplicates, it is slightly slower than `UNION ALL`.
- **`UNION ALL`**: Combines result sets and **includes all rows**, including duplicates. Use it when you know there are no duplicates or when you explicitly want to keep them. It is faster because it doesn't perform the duplicate check.

Syntax

```
SELECT column1, column2 FROM table1
[WHERE condition]
UNION / UNION ALL
SELECT column1, column2 FROM table2
[WHERE condition];
```

Example

Let's say we have two tables, `Developers` and `Designers`.

Table: `Developers`

EmployeeID	Name
101	Alice
102	Bob
105	Charlie

Table: `Designers`

EmployeeID	Name
201	Dave
202	Eve
105	Charlie

Notice that ‘Charlie’ exists in both tables.

Using `UNION` (Removes Duplicates)

```
SELECT EmployeeID, Name FROM Developers
UNION
SELECT EmployeeID, Name FROM Designers;
```

Result: ‘Charlie’ appears only once.

EmployeeID	Name
101	Alice
102	Bob
105	Charlie
201	Dave
202	Eve

Using `UNION ALL` (Keeps Duplicates)

```
SELECT EmployeeID, Name FROM Developers
UNION ALL
SELECT EmployeeID, Name FROM Designers;
```

Result: ‘Charlie’ appears twice.

EmployeeID	Name
101	Alice
102	Bob
105	Charlie
201	Dave
202	Eve
105	Charlie

GROUP BY with ROLLUP

`ROLLUP` is a powerful subclause for `GROUP BY` that allows you to generate multiple levels of aggregation in a single query. It provides subtotals for each level of the hierarchy specified in the `GROUP BY` clause, plus a grand total.

Intuitive Explanation

Imagine you have a spreadsheet with sales data organized by `Country`, then `City`, then `Product`.

- A simple `GROUP BY Country, City, Product` would give you the sales for each specific product in each specific city.
- Adding `ROLLUP` is like telling the database: "Okay, after you do that, please also add subtotal rows for me. Give me a subtotal for each city (summing up its products), then roll up and give me a subtotal for each country (summing up its cities), and finally, roll up one last time to give me the grand total of everything."

`ROLLUP(Country, City, Product)` generates aggregations for:

1. `(Country, City, Product)` - The most detailed level
2. `(Country, City)` - Subtotal
3. `(Country)` - Subtotal
4. `()` - The Grand Total

The database uses `NULL` as a placeholder to indicate “all values” for a column that has been “rolled up.”

Practical Example & Question

Let’s use the provided database schema to answer a common type of reporting question.

> **Question**
>
> Produce a list of the total number of slots booked per facility per month in the year of 2012. In this version, include output rows containing totals for all months per facility, and a total for all months for all facilities. The output table should consist of facility id, month and slots, sorted by the id and month. When calculating the aggregated values for all months and all facids, return `NULL` values in the month and facid columns.
>
> **Credits**: This question is adapted from PGExercises.

We can solve this using the concise `ROLLUP` function or the more verbose `UNION ALL` method.

Solution 1: Using `ROLLUP`

This approach is elegant and efficient. We specify the hierarchy (`facid`, then `month`) inside the `ROLLUP` clause, and it automatically generates the required subtotals and the grand total.

```sql
SELECT
    facid,
    EXTRACT(MONTH FROM starttime) AS month,
    SUM(slots) AS slots
FROM
    cd.bookings
WHERE
    EXTRACT(YEAR FROM starttime) = 2012
GROUP BY
    ROLLUP(facid, month) -- Create a hierarchy of groupings
ORDER BY
    facid,
    month;
```

How it works:

1. **`WHERE EXTRACT(YEAR FROM starttime) = 2012`**: Filters the data to only include bookings from the year 2012.

2. **`GROUP BY ROLLUP(facid, month)`**: This is the key instruction. It tells PostgreSQL to perform aggregations at three levels:
    - By `(facid, month)`: The total slots for each facility in each month.
    - By `(facid)`: It "rolls up" the month, giving a `NULL` for the `month` column and providing the total slots for the entire year for that facility.
    - By `()`: It "rolls up" both `facid` and `month`, giving `NULL` for both columns and providing the grand total slots for all facilities for the entire year.
3. **`ORDER BY facid, month`**: Sorts the results, ensuring the subtotal row for each facility appears right after its monthly data.

Solution 2: Using `UNION ALL`

This method achieves the same result by writing three separate queries and stitching them together. It's more explicit but much more verbose.

```sql
-- Query 1: Get the detailed monthly bookings per facility
SELECT
    facid,
    EXTRACT(MONTH FROM starttime) AS month,
    SUM(slots) AS slots
FROM
    cd.bookings
WHERE
    EXTRACT(YEAR FROM starttime) = 2012
GROUP BY
    facid, month

UNION ALL

-- Query 2: Get the subtotal for each facility (across all
        months)
SELECT
    facid,
    NULL AS month, -- Month column is NULL for the facility
        subtotal
    SUM(slots) AS slots
FROM
    cd.bookings
WHERE
    EXTRACT(YEAR FROM starttime) = 2012
GROUP BY
    facid
```

```sql
UNION ALL

-- Query 3: Get the grand total (across all facilities and
        months)
SELECT
    NULL AS facid,
        -- Both facid and month are NULL for the grand total
    NULL AS month,
    SUM(slots) AS slots
FROM
    cd.bookings
WHERE
    EXTRACT(YEAR FROM starttime) = 2012
ORDER BY
    facid,
    month;
```

How it works:

1. **First `SELECT`**: Calculates the base-level aggregation: total slots per facility, per month.
2. **Second `SELECT`**: Calculates the first level of subtotals: total slots per facility. We explicitly set `month` to `NULL` to match the required output.
3. **Third `SELECT`**: Calculates the grand total. We explicitly set both `facid` and `month` to `NULL`.
4. **`UNION ALL`**: Combines the results of these three independent queries into a single result set.
5. **`ORDER BY`**: Sorts the final, combined result set.

As you can see, `ROLLUP` provides a much cleaner and more maintainable way to generate summary rows compared to the manual `UNION ALL` approach.

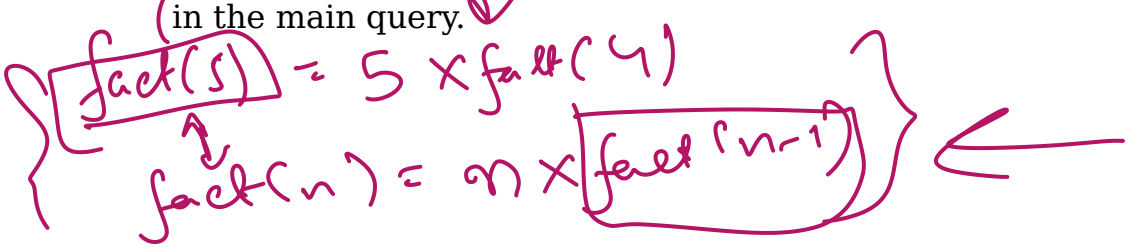
Common Table Expressions (CTEs)

WITH Clause

A CTE is a temporary, named result set that you can reference within a `SELECT`, `INSERT`, `UPDATE`, or `DELETE` statement. It helps improve readability and break down complex queries into logical, manageable steps.

Why We Use CTEs

- **Readability**: They make long and complex queries easier to read by separating different logical parts.
- **Modularity**: You can define a CTE once and reference it multiple times in the main query.

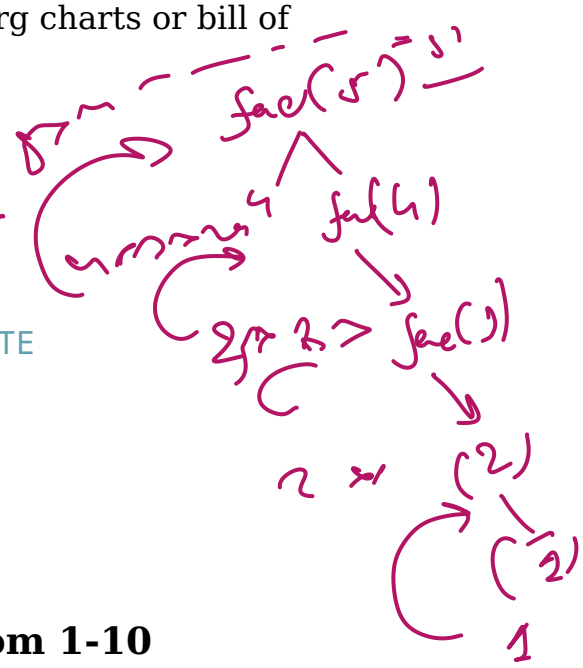


- **Recursion**: They are the only way to write recursive queries in SQL, which is essential for hierarchical data (like org charts or bill of materials).

WITH Clause Syntax

A CTE is defined using the `WITH` clause.

```
WITH CteName (column1, column2, ...) AS (
    -- This is the query that defines the CTE
    SELECT ...
    FROM ...
)
-- This is the main query that uses the CTE
SELECT * FROM CteName;
```



Simple Example: Generate Numbers from 1-10

While simple CTEs often select from existing tables, generating a sequence of numbers is a classic introduction to the power of **recursive CTEs**, which we will cover next. Here is how you do it:

```
WITH NumberSeries AS (
    -- 1. Anchor Member: The starting point of the recursion
    SELECT 1 AS MyNumber
    UNION ALL

    -- 2. Recursive Member: The part that calls itself
    SELECT MyNumber + 1
    FROM NumberSeries
    WHERE MyNumber < 10 -- 3. Termination Condition: Stops the
        recursion
)
-- 4. Main Query: Select from the generated CTE
SELECT MyNumber
FROM NumberSeries;
```

Result:

MyNumber
1
2

MyNumber
3
4
5
6
7
8
9
10

Recursive CTEs

A recursive CTE is one that references itself. It's composed of two main parts:

1. **Anchor Member**: A `SELECT` statement that runs once and provides the initial result set. It does not reference the CTE itself.
2. **Recursive Member**: A `SELECT` statement that references the CTE and is run repeatedly until it returns no more rows. These two members are combined using `UNION ALL`.

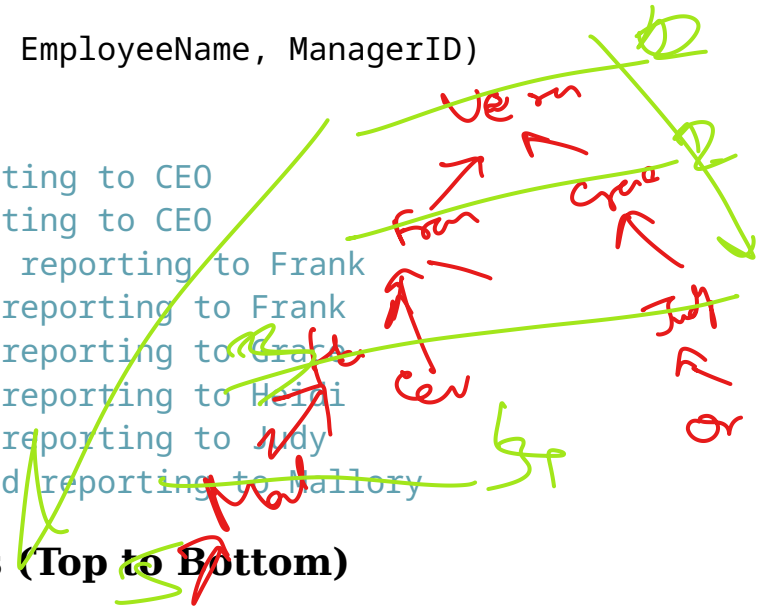
Setup: `Employees` Table

First, let's create and populate an `Employees` table that represents an organization's hierarchy. The `ManagerID` for the top-level employee (the CEO) is `NULL`.

```
-- Table Initialization
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    EmployeeName VARCHAR(50),
    ManagerID INT
```

```
);

-- Fill table with some values
INSERT INTO Employees (EmployeeID, EmployeeName, ManagerID)
        VALUES
(1, 'Vera', NULL),      -- CEO
(2, 'Frank', 1),        -- VP reporting to CEO
(3, 'Grace', 1),        -- VP reporting to CEO
(4, 'Heidi', 2),        -- Director reporting to Frank
(5, 'Ivan', 2),         -- Director reporting to Frank
(6, 'Judy', 3),         -- Director reporting to Grace
(7, 'Mallory', 4),      -- Manager reporting to Heidi
(8, 'Oscar', 6),        -- Manager reporting to Judy
(9, 'Peggy', 7);        -- Team Lead reporting to Mallory
```



Example 1: Finding All Reports (Top to Bottom)

Here, we want to find the entire reporting chain for a top-level manager (Vera, `EmployeeID = 1`). This is a "top-down" traversal of the hierarchy.

The Query

```
WITH RECURSIVE EmployeeHierarchy AS (

        -- Anchor Member: Select the top-level manager. Their
        manager is NULL.
    SELECT
        EmployeeID,
        EmployeeName,
        CAST(NULL AS VARCHAR(100)) AS ManagerName, -- The top-
        level manager has no manager
        1 AS Level
    FROM Employees
    WHERE EmployeeID = 1  -- Starting with Vera

    UNION ALL

    -- Recursive Member: Find employees who report to the people
        from the previous step.
    SELECT
        e.EmployeeID,
        e.EmployeeName,
        eh.EmployeeName AS ManagerName, -- The manager is the
        employee from the previous level (eh)
        eh.Level + 1
```

```
    FROM Employees e
    INNER JOIN EmployeeHierarchy eh ON e.ManagerID =
        eh.EmployeeID
    -- Join to find direct reports
)
-- Final SELECT to display the results with aliased column names
SELECT
    EmployeeID AS emp_id,
    EmployeeName AS emp_name,
    ManagerName AS manager_name,
    Level AS level
FROM EmployeeHierarchy;
```

Breakdown

1. **Anchor Member**: Selects the starting employee (`EmployeeID = 1`) and assigns them `Level = 1`.
2. **`UNION ALL`**: Combines the anchor result with the results of the recursive steps.
3. **Recursive Member**:
   - It joins the `Employees` table (`e`) with the CTE itself (`eh`).
   - The join condition `e.ManagerID = eh.EmployeeID` finds all employees (`e`) whose manager is already in the `EmployeeHierarchy` result set from the previous step.
   - `eh.Level + 1` increments the hierarchy level for each new layer of reports.
4. **Termination**: The recursion stops automatically when the recursive member finds no new employees to add (i.e., it reaches the employees at the bottom of the hierarchy who manage no one).

Expected Output:

This will show the entire reporting structure starting from `EmployeeID = 1`, including the name of each employee's direct manager.

emp_id	emp_name	manager_name	level
1	Vera	NULL	1
2	Frank	Vera	2
3	Grace	Vera	2

emp_id	emp_name	manager_name	level
4	Heidi	Frank	3
5	Ivan	Frank	3
6	Judy	Grace	3
7	Mallory	Heidi	4
8	Oscar	Judy	4
9	Peggy	Mallory	5

Example 2: Finding the Management Chain (Bottom to Top)

Now, let's do the opposite. We'll start with a specific employee (Peggy, `EmployeeID = 9`) and find their entire management chain all the way up to the CEO. This is a "bottom-up" traversal.

The Query

```
WITH RECURSIVE ManagerHierarchy AS (
    -- Anchor Member: Select the starting employee
    SELECT
        EmployeeID,
        EmployeeName,
        ManagerID,
        1 AS Level
    FROM Employees
    WHERE EmployeeID = 9 -- Starting with Peggy

    UNION ALL

    -- Recursive Member: Find the manager of the person in the
        previous step
    SELECT
        e.EmployeeID,
        e.EmployeeName,
        e.ManagerID,
```

```
        mh.Level + 1
    FROM Employees e
    INNER JOIN ManagerHierarchy mh ON e.EmployeeID = mh.ManagerID
    -- The join condition walks UP the chain
)
-- Final SELECT to display results
-- Join the completed hierarchy (mh) back to the Employees table
        (m) to get manager names
SELECT
    mh.EmployeeID AS emp_id,
    mh.EmployeeName AS emp_name,
    m.EmployeeName AS manager_name,
    mh.Level as bottom_to_top_level
FROM ManagerHierarchy mh
LEFT JOIN Employees m ON mh.ManagerID = m.EmployeeID
-- LEFT JOIN to handle the CEO whose ManagerID is NULL
ORDER BY mh.Level;
```

Breakdown

The logic is similar, but the join condition in the recursive member is flipped:

- `e.EmployeeID = mh.ManagerID`: This finds the employee (`e`) who is the manager of the person currently in the `ManagerHierarchy` (`mh`). This allows us to "walk up" the chain.

Expected Output:

This will show the full management chain for `EmployeeID = 9` (Peggy), all the way up to the CEO.

emp_id	emp_name	manager_name	bottom_to_top_level
9	Peggy	Mallory	1
7	Mallory	Heidi	2
4	Heidi	Frank	3
2	Frank	Vera	4
1	Vera	NULL	5